STRIDEMAP.AI
AgentGuard — AI Agent Governance Library

AgentGuard
Functional Specification

Version 1.0 | April 2026

Document Title	AgentGuard Functional Specification
Version	1.0 (Initial Release)
Date	April 2026
Author	Stridemap.ai Engineering & Product Team
Classification	**CONFIDENTIAL**
Status	Draft — Pending Review
Source Documents	AgentGuard PRD v1 | Roadmap v1 | System Architecture v1 | System Design v1 | Test Plan v1

© 2026 Stridemap.ai — All Rights Reserved. This document is confidential and intended solely for authorized recipients.

Table of Contents

1. Introduction & Purpose

1.1 Purpose of This Document

This Functional Specification defines the complete behavioural contract for AgentGuard — an open-source Python library that provides a non-invasive AI agent governance layer for LangGraph multi-agent systems. It serves as the authoritative reference for engineering, product, QA, security, and compliance stakeholders. The document synthesises content from the AgentGuard Product Requirements Document (PRD), Roadmap, System Architecture, System Design, and Test Plan.

1.2 Document Scope

This specification covers:

- All functional requirements organised by module
- API contracts and integration specifications
- Data schemas, security requirements, and non-functional constraints
- Acceptance criteria by release phase (MVP, v1.0, v1.5, v2.0)
- Out-of-scope items, risks, and glossary

1.3 Intended Audience

Audience	How They Use This Document
Security Engineers / CISOs	Evaluate threat coverage, security requirements, and OWASP LLM Top 10 alignment
AI / Platform Engineers	Implement integrations, understand LangGraph hook mechanics and API contracts
Compliance Officers / Auditors	Verify GDPR / SOC 2 alignment, audit chain requirements, and RBAC controls
DevOps / Infrastructure Engineers	Understand deployment topology, Kubernetes / Helm packaging, Prometheus metrics
QA Engineers	Drive test case development from functional and acceptance criteria

1.4 References

- AgentGuard PRD v1 — Product vision, personas, release criteria
- AgentGuard Roadmap v1 — Sprint-by-sprint deliverables (Sprints 1–24)
- AgentGuard System Architecture v1 — 8-layer architecture, component topology
- AgentGuard System Design v1 — Functional & non-functional requirement tables, API contracts

- AgentGuard Test Plan v1 — 5-layer test framework, CI/CD pipeline, adversarial test cases

1.5 Version History

Version	Date	Author	Change Summary
1.0	April 2026	Stridemap.ai Team	Initial functional specification synthesised from all v1 source documents

2. Product Overview

2.1 Product Vision

AgentGuard is the "Palo Alto Prisma but for AI agents" — a purpose-built, open-source Python governance layer that lets enterprises deploy LangGraph multi-agent systems with confidence. It enforces security policies, detects prompt injection attacks, prevents data leakage, and maintains cryptographically verifiable audit trails — all without modifying a single line of agent code.

2.2 Core Value Proposition

Pillar	What AgentGuard Delivers
Non-Invasive Integration	Drop-in LangGraph BaseCallbackHandler subclass — zero agent code changes required
Policy-as-Code Governance	YAML-defined security rules evaluated in-memory at sub-millisecond latency
Data Loss Prevention	100% local Microsoft Presidio NLP — no PII ever leaves the customer environment
Prompt Injection Defence	Two-stage detection: fast regex (< 5 ms p99) + LLM classifier fallback
Tamper-Proof Audit Chain	SHA-256 cryptographic event chaining — verifiable forensic trail
Fail-Open Safety	Any AgentGuard failure MUST never block agent execution — mirrors NGFW bypass mode
VPC-Only Data Handling	No outbound connections — all processing stays within customer infrastructure

2.3 Architecture Summary

AgentGuard is structured as an 8-layer governance stack:

- Layer 1 — LangGraph Integration Layer: BaseCallbackHandler hooks (on_tool_start, on_tool_end)
- Layer 2 — Policy Engine: YAML-defined ALLOW / BLOCK / ALERT / BYPASS decision engine
- Layer 3 — DLP Engine: Microsoft Presidio entity detection and redaction
- Layer 4 — Prompt Injection Detector: Regex Stage 1 + LLM Stage 2 classifier
- Layer 5 — Audit Chain: SHA-256 cryptographic event chaining with SQLite → PostgreSQL storage
- Layer 6 — Async Event Queue: asyncio in-memory queue with < 5 ms p99 write latency
- Layer 7 — FastAPI REST Layer: JWT-authenticated REST API (Sprint 5+)

- Layer 8 — Dashboard & Monitoring: Streamlit (MVP) evolving to Next.js 14 with WebSocket

2.4 Release Phases

Phase	Timeline	Key Capabilities
MVP	Sprints 1–4	Interception layer, basic policy engine, DLP v1, prompt injection Stage 1, SQLite audit chain, Streamlit dashboard
v1.0	Sprints 5–8	FastAPI REST layer, JWT auth, PostgreSQL, SIEM/CEF webhook, Prometheus metrics, Policy-as-Code v2, LLM injection classifier
v1.5	Sprints 9–12	SAML 2.0 / OIDC SSO, RBAC enforcement, Redis caching, Kubernetes / Helm packaging, Next.js dashboard
v2.0	Sprints 13–24	Multi-agent federation, Anthropic / OpenAI native support, S3 audit archival, AI-powered threat analytics, multi-tenant SaaS

3. Users, Personas & Use Cases

3.1 Primary Personas

3.1.1 Security Engineer / CISO

The Security Engineer ("Alex") is responsible for ensuring AI agents comply with enterprise security policies. Alex needs to:

- Define and deploy Policy-as-Code rules in YAML without engineering assistance
- Monitor real-time alerts when the policy engine BLOCKs or ALERTs on tool calls
- View the cryptographic audit chain to verify no events have been tampered with
- Integrate AgentGuard alerts into existing SIEM infrastructure (Splunk, Datadog, Microsoft Sentinel)
- Generate compliance reports for SOC 2, GDPR, and internal audit requirements

3.1.2 AI / Platform Engineer

The AI Engineer ("Jordan") builds and maintains LangGraph multi-agent pipelines. Jordan needs to:

- Integrate AgentGuard with zero agent code modifications — one-line callback registration
- Configure DLP sensitivity levels and PII entity types per deployment environment
- Tune prompt injection detection sensitivity to reduce false positives in production
- Access the REST API to query audit events and policy decisions programmatically
- Monitor Prometheus metrics for governance overhead and latency impact

3.1.3 Compliance Officer / Auditor

The Compliance Officer ("Morgan") validates that AI agent behaviour meets regulatory requirements. Morgan needs to:

- Access read-only audit log views scoped to their role (auditor RBAC)
- Export audit events as CSV / JSON for external audit tools
- Verify audit chain integrity using the verify_chain() function
- Review DLP redaction logs to confirm PII was not transmitted to external tools

3.1.4 DevOps / Infrastructure Engineer

The DevOps Engineer ("Sam") operates the AgentGuard deployment. Sam needs to:

- Deploy AgentGuard via Helm chart with a single values.yaml configuration
- Configure database backend (SQLite for dev, PostgreSQL for prod)
- Scrape Prometheus /metrics endpoint for operational dashboards
- Manage horizontal pod autoscaling and resource limits in Kubernetes

3.2 Use Cases

UC-ID	Use Case	Description
UC-01	Real-Time Tool Call Interception	AgentGuard intercepts every on_tool_start event and evaluates the configured policy before the tool executes
UC-02	PII Redaction in Tool Outputs	DLP engine scans on_tool_end payloads, redacts PII entities, and writes a redaction audit event
UC-03	Prompt Injection Blocking	Two-stage detector flags injected payloads; BLOCK decisions halt the tool call and raise an alert
UC-04	SIEM Alert Forwarding	High-severity policy events are formatted as CEF and forwarded to the configured SIEM webhook endpoint
UC-05	Audit Chain Verification	Auditor calls verify_chain() to confirm no audit records have been deleted or modified
UC-06	Policy Hot-Reload	Security engineer updates YAML policy file; AgentGuard reloads rules without agent restart
UC-07	Dashboard RBAC	Admin users see all data; auditor users see logs only; viewer users see metrics summaries
UC-08	Compliance Export	Auditor exports filtered audit events as CSV or JSON for external compliance reporting

4. System Scope & Boundaries

4.1 In-Scope Capabilities

Area	In-Scope Capability
Integration	LangGraph BaseCallbackHandler subclassing; on_tool_start / on_tool_end hook interception; Python 3.10–3.12 support; ARM64 and AMD64 architectures
Policy Engine	YAML Policy-as-Code evaluation; ALLOW / BLOCK / ALERT / BYPASS decisions; tool-name, argument, output, and agent-ID rule conditions; hot-reload without restart
DLP	Microsoft Presidio local NLP; 15+ PII entity types; REDACT / ALERT / BLOCK enforcement actions; confidence-threshold tuning; custom entity support
Prompt Injection	Stage 1 regex pattern library (< 5 ms p99); Stage 2 LLM classifier (only when Stage 1 = SUSPICIOUS); quarantine mode; sensitivity levels (LOW / MEDIUM / HIGH)
Audit Chain	SHA-256 cryptographic chaining; immutable append-only events; verify_chain() integrity check; SQLite (MVP) → PostgreSQL 15 (v1) storage
REST API	FastAPI REST layer; JWT Bearer authentication; 10 documented endpoints across audit, policy, health, and SIEM domains
Dashboard	Streamlit MVP; Next.js 14 + WebSocket (v2); RBAC role-gated views; real-time event stream; CSV / JSON export
Observability	Prometheus /metrics endpoint; latency histograms; event counters; SIEM CEF webhook; alerting hooks
Deployment	pip install package; Helm chart for Kubernetes; Docker image; environment-specific configuration via YAML/env vars

4.2 Explicitly Out of Scope

Out-of-Scope Item	Rationale
Agent code modification	AgentGuard is strictly non-invasive; it never changes agent logic
Non-LangGraph frameworks (CrewAI, AutoGen)	MVP and v1 target LangGraph exclusively; multi-framework is a v2+ roadmap item
External PII scanning APIs	All DLP processing is 100% local via Presidio; no cloud NLP services
Remediation / auto-correction of agent outputs	AgentGuard detects and alerts; it does not rewrite or correct tool inputs/outputs

Agent training or fine-tuning	Runtime governance only; no ML training pipeline
General-purpose SIEM platform	AgentGuard forwards CEF events; it does not replace Splunk / Datadog
Billing / licensing enforcement	Open-source; no commercial licensing logic in v1
Real-time agent behaviour modification	BLOCK decisions halt the call; AgentGuard does not redirect or substitute tool responses

5. Functional Specifications by Module

5.1 Module 1 — Agent Monitor & Trace Viewer (Interceptor Layer)

The Interceptor Layer is the foundational component of AgentGuard. It subclasses LangGraph's BaseCallbackHandler to intercept tool calls without modifying agent code.

Req. ID	Requirement Name	Functional Specification
FR-AM-01	**Tool Call Interception**	AgentGuard SHALL intercept 100% of LangGraph tool calls via on_tool_start and on_tool_end callback hooks. Interception MUST be non-invasive — no agent source code modifications are permitted. The interceptor MUST register as a LangGraph callback handler using a single-line integration call.
FR-AM-02	**Event Payload Capture**	For every intercepted event, AgentGuard SHALL capture: agent_id, tool_name, tool_input (full payload), tool_output (full payload), timestamp (ISO-8601 UTC), execution_duration_ms, decision (ALLOW/BLOCK/ALERT/BYPASS), and session_id. All fields are mandatory for audit events.
FR-AM-03	**Fail-Open Guarantee**	Any exception or error within AgentGuard's interception or governance pipeline SHALL NOT block agent execution. The system MUST catch all internal exceptions, log them to the error stream, and return BYPASS to the agent. This mirrors Palo Alto NGFW fail-open (bypass) mode.

5.1.1 Behaviour Specification: Fail-Open Mode

When AgentGuard encounters an internal failure (policy load error, DLP engine crash, audit queue overflow), it MUST:

- Log the error with full stack trace to stderr and the configured error log
- Return decision = BYPASS to the calling agent immediately
- Increment the agentguard_errors_total Prometheus counter
- NOT raise an exception to the agent runtime
- Continue processing subsequent tool calls normally once the failure condition clears

5.2 Module 2 — Policy Engine & DLP Interceptor

The Policy Engine evaluates YAML-defined rules against intercepted tool call payloads and produces governance decisions. The DLP Interceptor scans payloads for PII and sensitive data using Microsoft Presidio.

Req. ID	Requirement Name	Functional Specification
FR-PE-01	**YAML Policy Loading**	The Policy Engine SHALL load rules from one or more YAML files specified in the AgentGuard configuration. Rules MUST be validated against the canonical schema at load time. Invalid rules MUST cause a validation error with a descriptive message and SHALL NOT be applied.
FR-PE-02	**Policy Decision Engine**	For each intercepted tool call, the Policy Engine SHALL evaluate all applicable rules in priority order and produce exactly one decision: ALLOW (permit execution), BLOCK (halt execution, raise alert), ALERT (permit but log high-severity event), or BYPASS (skip all checks, used for trusted tools). Decision MUST complete within 10 ms p99.
FR-PE-03	**Rule Conditions**	Rules SHALL support conditions on: tool_name (exact match, wildcard, regex), tool_input fields (key/value, regex), tool_output fields, agent_id, and environment tags. Multiple conditions within a rule SHALL be evaluated as logical AND. Rules within a policy file SHALL be evaluated as logical OR (first match wins).
FR-PE-04	**Policy Hot-Reload**	The Policy Engine SHALL support hot-reload of YAML policy files without requiring an agent restart or AgentGuard reinitialisation. Hot-reload MUST be atomic — the engine MUST continue using the old policy until the new policy has been fully validated and swapped in memory.
FR-PE-05	**DLP PII Detection**	The DLP engine SHALL scan all tool_input and tool_output payloads using Microsoft Presidio running 100% locally. Detected PII entities SHALL include at minimum: PERSON,

		EMAIL_ADDRESS, PHONE_NUMBER, CREDIT_CARD, IBAN_CODE, SSN, IP_ADDRESS, DATE_TIME, NRP, LOCATION, MEDICAL_LICENSE, US_BANK_NUMBER, US_DRIVER_LICENSE, US_PASSPORT.
FR-PE-06	**DLP Enforcement Actions**	For each detected PII entity, the DLP engine SHALL apply the configured enforcement action: REDACT (replace entity text with [REDACTED_TYPE]), ALERT (log a high-severity event without redaction), or BLOCK (halt tool call and raise alert). The confidence threshold for triggering actions SHALL be configurable (default: 0.85).

5.2.1 YAML Policy Schema

The canonical AgentGuard YAML policy schema is structured as follows:

```
version: "1.0"rules:  - id: RULE-001    name: Block External HTTP Calls
priority: 10    enabled: true    conditions:      tool_name: "http_request"
tool_input.url:        pattern: "^https?://(?!internal\.)"    action: BLOCK
severity: HIGH    message: "External HTTP calls are not permitted"  - id: RULE-002
name: Alert on File Write    priority: 20    conditions:      tool_name:
"file_write"    action: ALERT    severity: MEDIUM
```

5.3 Module 3 — Prompt Injection Detector

The Prompt Injection Detector uses a two-stage architecture to identify adversarial inputs that attempt to override agent instructions, exfiltrate data, or manipulate tool behaviour.

Req. ID	Requirement Name	Functional Specification
FR-PI-01	**Stage 1 Regex Detection**	The detector SHALL evaluate all tool_input payloads against a maintained library of prompt injection regex patterns. Stage 1 MUST complete within 5 ms p99. Stage 1 produces three outputs: CLEAN (no match), SUSPICIOUS (partial match, escalate to Stage 2), or INJECTED (high-confidence match, immediately BLOCK).
FR-PI-02	**Stage 2 LLM Classifier**	When Stage 1 produces SUSPICIOUS, the detector SHALL escalate to an LLM-based classifier. The classifier MUST be configured with an internal or locally-hosted LLM endpoint (no external API calls in VPC-only mode).

		Stage 2 returns INJECTED or CLEAN. The Stage 2 result is authoritative for the final decision.
FR-PI-03	**Quarantine Mode & Sensitivity**	The detector SHALL support three sensitivity levels: LOW (Stage 1 only, INJECTED = BLOCK), MEDIUM (Stage 1 + Stage 2, INJECTED = BLOCK), HIGH (Stage 1 + Stage 2, SUSPICIOUS = BLOCK). Quarantine mode SHALL isolate detected payloads for human review without immediately blocking the agent session.

5.4 Module 4 — Tamper-Proof Audit Chain

The Audit Chain provides cryptographically verifiable, append-only records of all governance events. It ensures that no audit record can be deleted, modified, or reordered without detection.

Req. ID	Requirement Name	Functional Specification
FR-AC-01	**Cryptographic Event Chaining**	Each audit event SHALL contain a chain_hash computed as: chain_hash = SHA-256(previous_chain_hash || event_hash), where event_hash = SHA-256(event_payload_JSON). The first event in a chain uses a genesis_hash of 64 zero bytes as the previous_chain_hash.
FR-AC-02	**Append-Only Storage**	The audit store SHALL be strictly append-only. No UPDATE or DELETE operations SHALL be permitted on audit event records at the database layer. The AgentGuard schema MUST enforce this constraint via database-level triggers or application-layer controls.
FR-AC-03	**Integrity Verification**	AgentGuard SHALL expose a verify_chain() function that replays the entire chain from genesis, recomputes each chain_hash, and returns: VALID (all hashes match), TAMPERED (one or more hashes do not match, with the index of the first mismatch), or INCOMPLETE (missing events detected).

FR-AC-04	**Audit Event Schema**	Every audit event SHALL contain the mandatory fields: event_id (UUIDv4), event_type, timestamp (ISO-8601 UTC), agent_id, session_id, tool_name, tool_input, tool_output, decision, severity, chain_hash, previous_chain_hash, and metadata (extensible JSON object).

5.5 Module 5 — Dashboard & Audit Export

The Dashboard provides real-time visibility into agent governance activity, policy decisions, and audit events. Access is gated by RBAC role assignments.

Req. ID	Requirement Name	Functional Specification
FR-DB-01	**Real-Time Event Stream**	The Dashboard SHALL display audit events in real-time using WebSocket (v2) or polling (MVP). The event stream SHALL be filterable by: time range, agent_id, tool_name, decision (ALLOW/BLOCK/ALERT/BYPASS), severity (LOW/MEDIUM/HIGH/CRITICAL), and event_type.
FR-DB-02	**RBAC Role-Gated Views**	The Dashboard SHALL enforce RBAC with three roles: ADMIN (full access: events, policies, users, settings, exports), AUDITOR (read-only: events, audit chain verification, exports), VIEWER (metrics and summary dashboards only — no raw event access). Role assignment is managed via the REST API by ADMIN users.

5.5.1 Dashboard Views by Role

Dashboard View	ADMIN	AUDITOR	VIEWER
Live Event Stream	Yes	Yes	No
Policy Decision Summary	Yes	Yes	Summary only
DLP Redaction Log	Yes	Yes	No
Prompt Injection Alerts	Yes	Yes	Count only
Audit Chain Verification	Yes	Yes	No

Policy Management (CRUD)	Yes	No	No
User & Role Management	Yes	No	No
CSV / JSON Export	Yes	Yes	No
Prometheus Metrics	Yes	Yes	Yes

6. API Functional Specification

AgentGuard exposes a FastAPI REST layer (Sprint 5+) with JWT Bearer authentication. All API endpoints are prefixed with /api/v1. Unauthenticated requests MUST receive HTTP 401. Requests with insufficient RBAC permissions MUST receive HTTP 403.

6.1 Authentication

- Authentication: JWT Bearer tokens — all endpoints require Authorization: Bearer <token> header
- Token issuance: POST /api/v1/auth/token — accepts username/password, returns access_token and expires_in
- Token refresh: POST /api/v1/auth/refresh — returns a new access token given a valid refresh token
- Token expiry: Access tokens expire in 15 minutes; refresh tokens in 24 hours
- SAML 2.0 / OIDC SSO: Sprint 9+ — tokens issued after successful IdP assertion

6.2 API Endpoint Catalogue

Endpoint	Method	Functional Specification
GET /api/v1/health	GET	Returns system health status. No authentication required. Response includes: status (ok/degraded/down), version, uptime_seconds, database connectivity, and policy engine status. Used by Kubernetes liveness and readiness probes.
GET /api/v1/audit/events	GET	Returns paginated list of audit events. Query parameters: page, page_size (max 100), start_time, end_time, agent_id, tool_name, decision, severity. Response: { events: [...], total, page, page_size }. Requires AUDITOR or ADMIN role.
GET /api/v1/audit/events/{id}	GET	Returns a single audit event by event_id (UUIDv4). Includes all mandatory event fields plus chain_hash and previous_chain_hash for verification. HTTP 404 if event not found. Requires AUDITOR or ADMIN role.
POST /api/v1/audit/verify	POST	Triggers verify_chain() across the specified range (default: full chain). Body: { start_index: int, end_index: int }. Response: { result: VALID|TAMPERED|INCOMPLETE, first_mismatch_index: int|null, events_checked: int }. Requires AUDITOR or ADMIN role.
GET /api/v1/audit/export	GET	Exports audit events as CSV or JSON. Query parameters: format (csv|json), start_time, end_time, agent_id, decision. Returns file download response.

		Max export size: 100,000 events per request. Requires AUDITOR or ADMIN role.
GET /api/v1/policies	GET	Returns list of all loaded policy files and their rule counts, enabled status, and last-loaded timestamp. Requires ADMIN role.
POST /api/v1/policies	POST	Uploads a new YAML policy file. Body: multipart/form-data with policy_file field. AgentGuard validates the policy schema before accepting. Returns: { policy_id, rules_loaded, validation_errors: [] }. Triggers hot-reload. Requires ADMIN role.
POST /api/v1/policies/reload	POST	Triggers an immediate hot-reload of all policy files from their configured source paths. Returns: { reloaded: true, policies_loaded: int, reload_time_ms: int }. Requires ADMIN role.
GET /api/v1/siem/config	GET	Returns current SIEM webhook configuration (endpoint URL, severity threshold, event_types). Webhook secret is NEVER returned in API responses. Requires ADMIN role.
POST /api/v1/siem/test	POST	Sends a synthetic CEF test event to the configured SIEM webhook endpoint. Returns: { sent: true, response_code: int, response_time_ms: int } or { sent: false, error: string }. Requires ADMIN role.

7. Integration Specifications

7.1 SIEM Integration (CEF Webhook)

AgentGuard forwards high-severity governance events to external SIEM platforms using the Common Event Format (CEF) over HTTPS webhook. Supported platforms: Splunk, Datadog, and Microsoft Sentinel.

Configuration Parameter	Specification
webhook_url	HTTPS endpoint of the SIEM receiver. Must be reachable from within the VPC. No plaintext HTTP permitted.
webhook_secret	HMAC-SHA256 secret for payload signature. Sent in X-AgentGuard-Signature header. Stored encrypted at rest; never returned by the API.
severity_threshold	Minimum severity level for SIEM forwarding. Values: LOW, MEDIUM, HIGH, CRITICAL. Default: HIGH.
event_types	Comma-separated list of event types to forward. Values: BLOCK, ALERT, INJECTION_DETECTED, DLP_VIOLATION, CHAIN_TAMPERED. Default: all.
retry_policy	3 retries with exponential backoff (1s, 2s, 4s). Failed webhooks are queued for up to 1 hour before being discarded and logged.

7.1.1 CEF Event Format

```
CEF:0|Stridemap.ai|AgentGuard|1.0|POLICY_BLOCK|Tool call blocked by
policy|8|rt=<epoch_ms> src=<agent_id> dhost=<tool_name> act=BLOCK
reason=<rule_id>:<rule_name> cs1=<session_id> cs1Label=session_id cs2=<chain_hash>
cs2Label=chain_hash
```

7.2 SSO Integration (SAML 2.0 / OIDC)

Sprint 9+ introduces SSO integration for enterprise authentication. AgentGuard supports both SAML 2.0 and OIDC (OpenID Connect) identity providers.

Parameter	Specification
SAML 2.0 SP metadata	AgentGuard acts as SAML Service Provider. Metadata endpoint: GET /api/v1/auth/saml/metadata
OIDC discovery	Supports OIDC Discovery Document (/.well-known/openid-configuration) from the configured IdP
Role mapping	IdP group claims are mapped to AgentGuard RBAC roles (ADMIN / AUDITOR / VIEWER) via configuration
Session duration	Configurable; default 8 hours. JWT refresh tokens issued after SSO assertion

7.3 Prometheus Metrics Integration

AgentGuard exposes a Prometheus-compatible /metrics endpoint for operational observability.

Metric Name	Type	Description
agentguard_tool_calls_total	Counter	Total tool calls intercepted, labelled by decision and tool_name
agentguard_policy_eval_duration_ms	Histogram	Policy evaluation latency in milliseconds (p50, p95, p99)
agentguard_dlp_detections_total	Counter	DLP PII entity detections, labelled by entity_type and action
agentguard_injection_detections_total	Counter	Prompt injection detections by stage and result
agentguard_audit_queue_depth	Gauge	Current async event queue depth — alert if > 1000
agentguard_audit_write_duration_ms	Histogram	Audit event write latency; target < 5 ms p99
agentguard_errors_total	Counter	Total internal errors triggering fail-open, labelled by error_type
agentguard_siem_webhook_duration_ms	Histogram	SIEM webhook delivery latency; labelled by status_code

8. Data Functional Specification

8.1 Core Audit Event Schema

Field	Type	Specification
event_id	UUID v4	Globally unique identifier for the event. Immutable once written.
event_type	Enum	TOOL_CALL_START, TOOL_CALL_END, POLICY_DECISION, DLP_DETECTION, INJECTION_DETECTED, CHAIN_VERIFIED, ERROR
timestamp	ISO-8601	UTC timestamp of event creation. Precision: milliseconds.
agent_id	String	Identifier of the LangGraph agent that triggered the event.
session_id	UUID v4	Identifier for the agent execution session.
tool_name	String	Name of the LangGraph tool being called.
tool_input	JSON	Full tool input payload. PII redacted if DLP action = REDACT.
tool_output	JSON	Full tool output payload. PII redacted if DLP action = REDACT. Null for TOOL_CALL_START events.
decision	Enum	ALLOW, BLOCK, ALERT, BYPASS
severity	Enum	LOW, MEDIUM, HIGH, CRITICAL
chain_hash	String (64 hex)	SHA-256 hash: SHA-256(previous_chain_hash || event_hash)
previous_chain_hash	String (64 hex)	chain_hash of the immediately preceding audit event. Genesis event uses 64 zeros.
metadata	JSON	Extensible metadata object: rule_id, dlp_entities, injection_stage, error_type, etc.

8.2 Storage Evolution

Phase	Storage Backend	Specification
MVP	SQLite	Local file-based SQLite database. Single-node only. Suitable for development and single-agent deployments. Max recommended: 10M events.

v1.0	PostgreSQL 15	Production-grade relational database with connection pooling (PgBouncer). Supports concurrent multi-agent deployments. Enforces append-only via DB triggers.
v1.5	PostgreSQL + Redis	Redis for policy cache and session state. Reduces policy evaluation latency. Cache TTL: 30 seconds.
v2.0	PostgreSQL + Redis + S3	S3-compatible object storage for audit archive. Events older than configurable retention window are migrated to S3. Supports multi-tenant partitioning.

9. Security Functional Requirements

9.1 OWASP LLM Top 10 Threat Coverage

Threat ID	OWASP LLM Threat	AgentGuard Mitigation
T1	Prompt Injection	Two-stage detector (regex + LLM classifier); quarantine mode; configurable sensitivity
T2	Data Exfiltration via Tool Calls	Policy Engine BLOCK rules on external HTTP / file-write tool calls; DLP scanning of all outputs
T3	Sensitive Data Disclosure	Presidio DLP with 15+ PII entity types; REDACT enforcement replaces PII before audit logging
T4	Insecure Output Handling	DLP engine scans tool_output payloads; BLOCK decision halts downstream agent steps
T5	Supply Chain Attacks on Plugins	Policy rules can whitelist approved tool names; unknown tools trigger ALERT
T6	Excessive Agency / Privilege Escalation	Policy Engine enforces least-privilege tool access; BLOCK on unauthorised tool combinations
T7	Model Denial of Service	Rate-limiting policy rules; alert on anomalous call frequency (v1.5+)
T8	Insecure Plugin Design	All tool calls intercepted regardless of plugin source; policy rules apply universally
T9	Audit & Traceability Gaps	SHA-256 cryptographic audit chain; verify_chain() integrity check; SIEM forwarding

9.2 Security Non-Negotiables

The following security requirements are absolute constraints with no exceptions:

- VPC-Only Data Handling: No data SHALL leave the customer VPC. All processing is local.
- No External NLP APIs: DLP processing uses only locally-deployed Microsoft Presidio.
- Fail-Open Under Failure: AgentGuard failures MUST NEVER block agent execution.
- Audit Append-Only: No UPDATE or DELETE operations are permitted on audit event records.
- Cryptographic Integrity: SHA-256 chain_hash computation cannot be bypassed or disabled.
- Encryption at Rest: All audit data MUST be stored with AES-256 encryption at rest.
- TLS in Transit: All API and webhook communication MUST use TLS 1.2 or higher.
- Webhook Secret Confidentiality: SIEM webhook secrets are NEVER returned by any API endpoint.

10. Non-Functional Requirements

NFR ID	Requirement	Target / Specification
NFR-01	**Interception Latency**	Policy evaluation overhead MUST NOT exceed 10 ms p99 per tool call
NFR-02	**Audit Write Latency**	Async event queue write latency MUST be < 5 ms p99
NFR-03	**Prompt Injection Stage 1**	Regex detection MUST complete within 5 ms p99
NFR-04	**DLP Throughput**	DLP engine SHALL process payloads up to 10 KB within 50 ms p99
NFR-05	**System Availability**	AgentGuard daemon target: 99.9% uptime (8.7 hours downtime/year). Fail-open guarantees zero additional agent downtime.
NFR-06	**Scalability**	Must support 100+ concurrent agents in v1.0; 1000+ in v2.0 via horizontal pod autoscaling
NFR-07	**Data Retention**	Audit events retained for minimum 90 days in hot storage; configurable archival to S3 (v2.0)
NFR-08	**Encryption at Rest**	All audit data encrypted at rest using AES-256
NFR-09	**Encryption in Transit**	All API and webhook traffic MUST use TLS 1.2 or higher. Plaintext HTTP is not permitted.
NFR-10	**Python Compatibility**	Python 3.10, 3.11, and 3.12 on ARM64 (M1/M2) and AMD64 architectures
NFR-11	**Memory Footprint**	AgentGuard daemon idle memory MUST be < 256 MB. Policy engine in-memory cache < 64 MB per 1,000 rules.
NFR-12	**API Response Time**	REST API endpoints MUST respond within 200 ms p95 under normal load
NFR-13	**Observability**	All key metrics exposed via Prometheus /metrics. Structured JSON logging to stdout. Trace IDs propagated via OpenTelemetry.

11. Acceptance Criteria by Release Phase

11.1 MVP — Sprints 1–4

Exit Criteria — MVP is complete when all of the following are verified:

- BaseCallbackHandler subclass intercepts 100% of on_tool_start and on_tool_end events in a LangGraph test agent
- Policy Engine loads a YAML rule file and correctly produces ALLOW / BLOCK / ALERT / BYPASS decisions
- DLP engine detects at least 10 PII entity types and applies REDACT action on tool payloads
- Stage 1 prompt injection regex detects 90% of OWASP LLM Top 10 injection patterns in the test dataset
- Audit chain appends events to SQLite; verify_chain() returns VALID on unmodified chain and TAMPERED when a record is manually altered
- Streamlit dashboard displays live event stream with filter controls
- Integration test: Zero LangGraph agent failures caused by AgentGuard errors (fail-open verified)
- Performance: Policy evaluation overhead < 10 ms p99 on reference hardware (AWS t3.medium)

11.2 v1.0 — Sprints 5–8

Exit Criteria — v1.0 ships when all MVP criteria are met plus:

- FastAPI REST layer operational with JWT authentication; all 10 documented endpoints return correct responses
- PostgreSQL backend replaces SQLite in staging; all audit tests pass against PostgreSQL
- SIEM CEF webhook delivers events to a Splunk test instance within 5 seconds of event creation
- Prometheus /metrics endpoint exposes all 8 documented metrics
- Stage 2 LLM injection classifier integrated and reduces false-positive rate by >20% vs Stage 1 alone
- Policy hot-reload test: update YAML rules without restarting agent; new rules applied within 5 seconds
- REST API response time < 200 ms p95 under 50 concurrent requests in load test

11.3 v1.5 — Sprints 9–12

Exit Criteria — v1.5 ships when all v1.0 criteria are met plus:

- SAML 2.0 SSO integration verified against at least one enterprise IdP (Okta or Azure AD)
- RBAC enforcement: AUDITOR role cannot access policy management endpoints; VIEWER role cannot access raw event stream

- Redis cache reduces policy evaluation latency by >30% vs non-cached baseline in load test
- Helm chart deploys AgentGuard to a Kubernetes cluster in < 5 minutes from values.yaml configuration
- Next.js dashboard delivers real-time events via WebSocket with < 500 ms end-to-end latency

11.4 v2.0 — Sprints 13–24

Exit Criteria — v2.0 ships when all v1.5 criteria are met plus:

- Multi-agent federation: AgentGuard governs agent-to-agent tool calls across a LangGraph multi-agent graph
- S3 audit archival operational: events older than retention window automatically migrated
- AI-powered threat analytics surface anomalous tool call patterns with >80% precision
- Multi-tenant deployment: tenant data fully isolated at database and API layer
- Support for Anthropic and OpenAI native agent frameworks in addition to LangGraph

12. Out of Scope

The following capabilities are explicitly out of scope for AgentGuard v1.x. They may be considered in future versions.

Item	Target Version	Note
Support for non-LangGraph agent frameworks (CrewAI, AutoGen, LlamaIndex)	v2.0+	Framework-specific adapter layer planned for v2.0
Automated remediation or correction of agent outputs	Not planned	AgentGuard detects and alerts; it does not rewrite outputs
Cloud-based PII scanning APIs (AWS Comprehend, Google DLP)	Not planned	VPC-only constraint — all DLP is local Presidio
General-purpose SIEM platform functionality	Not planned	AgentGuard forwards events; it doesn't replace Splunk
Agent training, fine-tuning, or ML model management	Not planned	Runtime governance only
Billing, licensing, or commercial entitlement enforcement	Not planned	Open-source; no commercial licensing in v1
Real-time redirection or substitution of blocked tool calls	Not planned	BLOCK halts; AgentGuard does not redirect
Compliance certification (SOC 2 Type II, ISO 27001)	v2.0+	AgentGuard outputs support certification; certification itself is out of scope

13. Risks & Mitigations

ID	Risk	Severity	Mitigation
R1	LangGraph API changes break BaseCallbackHandler compatibility	**HIGH**	Pin to tested LangGraph versions in requirements.txt; maintain compatibility matrix; automate regression tests on LangGraph releases
R2	Presidio DLP model accuracy insufficient for enterprise deployment	**HIGH**	Run F1-score benchmarks on enterprise datasets; support custom Presidio recognisers; allow confidence threshold tuning
R3	Prompt injection Stage 2 LLM classifier introduces latency > 500 ms	**MEDIUM**	Stage 2 triggered only for SUSPICIOUS payloads; async classification with configurable timeout; fallback to Stage 1 result on timeout
R4	Async event queue overflow under high agent throughput	**MEDIUM**	Alert at 80% queue capacity; configurable queue size; shed non-critical events; emit Prometheus alert
R5	Policy YAML complexity leads to misconfiguration and missed threats	**MEDIUM**	Schema validation at load time; policy linter CLI tool; library of reference policy templates
R6	SHA-256 chain computation becomes a throughput bottleneck at scale	**LOW**	Benchmark at 1,000 events/sec; consider batched hashing for v2.0 high-throughput deployments
R7	Open-source adoption leads to undisclosed production deployments with no support	**LOW**	Provide clear documentation, security advisories channel, and opt-in telemetry for adoption analytics

14. Glossary

Term	Definition
AgentGuard	The open-source Python library providing AI agent governance for LangGraph systems
LangGraph	LangChain's framework for building stateful, multi-actor applications with LLMs using a directed graph execution model
BaseCallbackHandler	LangGraph's base class for event hook callbacks — AgentGuard subclasses this for non-invasive interception
on_tool_start / on_tool_end	LangGraph callback hooks fired before and after a tool call respectively
Policy-as-Code	The practice of defining security and governance rules in version-controlled YAML files rather than hard-coded logic
DLP (Data Loss Prevention)	Controls that detect and prevent unauthorised transmission of sensitive or personally identifiable data
Presidio	Microsoft's open-source PII detection and anonymisation framework, used locally within AgentGuard
ALLOW	Policy decision: permit the tool call to proceed without restriction
BLOCK	Policy decision: halt the tool call, return an error to the agent, and log a high-severity audit event
ALERT	Policy decision: permit the tool call but log a high-severity audit event and forward to SIEM
BYPASS	Policy decision: skip all AgentGuard checks for this tool call (used for trusted tools or fail-open)
Fail-Open	Design pattern where any governance system failure permits the underlying operation rather than blocking it — mirrors NGFW bypass mode
CEF (Common Event Format)	ArcSight-defined log format widely supported by SIEM platforms for event ingestion
chain_hash	SHA-256 digest linking each audit event to its predecessor, forming a tamper-evident chain
RBAC	Role-Based Access Control — controls what actions each user role (ADMIN / AUDITOR / VIEWER) is permitted to perform
SIEM	Security Information and Event Management — platforms such as Splunk, Datadog, and Microsoft Sentinel
SSO	Single Sign-On — federated authentication using SAML 2.0 or OIDC identity providers

VPC	Virtual Private Cloud — isolated network boundary within which all AgentGuard data processing occurs
OWASP LLM Top 10	The Open Worldwide Application Security Project's list of the ten most critical security risks for LLM applications
JWT (JSON Web Token)	Compact URL-safe token format used for stateless authentication in the AgentGuard REST API